Masrah V2 Refactoring Report

Clean Architecture, SOLID and Creational Design Patterns for the Masrah Flutter app

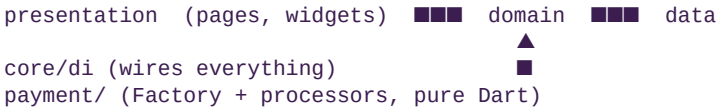
1. Code-smell report (legacy_masrah_app.dart)

The V1 app put UI, data parsing, pricing and payment routing into two widget classes. These are the main violations found.

#	Where in V1	Problem	Rule it breaks	Fixed in V2 by
1	_CheckoutScreenState.calculateGrandTotal()	Pricing, fees and VAT calculated inside a widget State.	Single Responsibility Principle; separation of concerns	CalculateTicketPrice UseCase (domain)
2	calculateGrandTotal(), handlePayment()	Long if / else-if chains on payment name strings. Adding a method means editing this code.	Open/Closed Principle	PaymentProcessor classes + PaymentMethodFactory
3	_EventListScreenState.fetchEventsFromApi()	JSON fetching and decoding written inside the widget; the screen creates its own low-level dependencies (jsonDecode, Future.delayed, Random).	Dependency Inversion Principle; Single Responsibility	EventRepository interface, LocalDataSource, ApiClient
4	eventsData (List), event['title'], eventData['base_price']	Raw Maps and dynamic values instead of typed objects. A typo in a key only fails at runtime.	Clean code: use meaningful types; avoid primitive obsession	Event / SeatCategory / Ticket entities + EventModel.fromJson
5	0.025, 0.02, 5, 0.14, 'Fawry', 'Credit Card' ...	Magic numbers and magic strings repeated through the code.	Clean code: no magic numbers; DRY	Named constants and enum PaymentType, one processor per rule set
6	calculateGrandTotal() vs payment_rules.md	Fees did not match the business rules (Fawry 2% + 5 EGP instead of tiers, Vodafone 1% + 3 instead of capped 1%, wallet flat 10) and VAT was taken on tickets plus fees instead of the base price only. No rounding to piastres.	Correctness; single source of truth	Each processor implements its rule; VAT on base; round2()
7	build() of CheckoutScreen (about 200 lines)	One huge build method mixing layout, state and inline logic.	Clean code: small functions; Single Responsibility	Smaller widgets (PriceSummaryCard, EventCard) and helper methods
8	handlePayment()	print() debugging calls, no input validation, no seat lock, no error handling, UI shows 'Success' for everything.	Clean code: no debug noise; fail fast; robust errors	BookTicketUseCase validates, checks limits, locks the seat and releases it on failure
9	Old JSON hardcoded inside fetchEventsFromApi()	Test data embedded in code and in the old API shape (location, base_price, vip_price).	Clean code: keep data out of logic	assets/events_api_response.json read through ApiClient and mapped by EventModel

2. New architecture

The project is split into layers. Dependencies only point inward toward the Domain layer.



Layer	Contains	Depends on
presentation	EventListPage, CheckoutPage, EventCard, PriceSummaryCard	domain, payment (pure Dart types)
domain	Event, SeatCategory, Ticket, BookingRequest(+Builder), EventRepository interface, 4 use cases	payment (pure Dart) only
data	EventModel, TicketModel, EventRepositoryImpl, LocalDataSource	domain, core/network
payment	PaymentType, PaymentProcessor, Factory, 4 processors, exceptions	nothing
core	ApiClient (Singleton), AppDependencies (DI)	all layers (composition root)

3. Design decisions

Factory Method. PaymentMethodFactory.create(type) returns the right PaymentProcessor. Each processor owns its fee, validation, limit and processing. The checkout screen never checks which method is selected: it loops over PaymentType.values and draws the inputs from processor.fields.

Singleton. ApiClient is created once (private constructor + factory constructor), so the whole app shares one client and its configuration.

Builder. BookingRequestBuilder needs only an event and a seat; quantity, extras (VIP parking, merchandise) and payment details are optional. build() creates an immutable BookingRequest.

Dependency injection. AppDependencies is the only place that uses *new* on repositories, data sources and use cases. main.dart passes ready-made use cases to the pages through constructors, so widgets never build their own dependencies.

Use cases. GetEventsUseCase, CalculateTicketPriceUseCase, GetPaymentFieldsUseCase and BookTicketUseCase. BookTicketUseCase validates input, checks the transaction limit, locks the seat *before* the processor runs, and releases the seat if payment fails.

Assumptions. (a) Fees, tier and limit checks use the base amount (seat price x quantity + extras) for the whole transaction. (b) Extras are flat prices per booking (100 and 200 EGP) and are part of the base. (c) Limits are checked against the final total. (d) Payment calls are simulated; Vodafone Cash polls every 5 seconds and the fake backend approves on the first poll.

4. Pricing examples (from the rules)

Case	Base	VAT 14%	Fee	Total (EGP)
Credit card, 1 x Omar Khairat Silver	1500.00	210.00	39.50	1749.50
Fawry, Tier 1 (450)	450.00	63.00	5.00	518.00
Fawry, Tier 2 (1500)	1500.00	210.00	10.00	1720.00
Fawry, Tier 3 (2500)	2500.00	350.00	25.00	2875.00
Vodafone Cash, 1200	1200.00	168.00	12.00	1380.00
Vodafone Cash, 4000 (fee capped)	4000.00	560.00	20.00	4580.00
Digital wallet, 1500	1500.00	210.00	22.50	1732.50

5. Screenshots of the working app

[Insert screenshots here: event list, checkout, order summary, success dialog]